A.10 Experiment 8: Details for low rank representations

For task loss matching, the low rank transformation is ensured by a bottleneck in the stitching layer: for a prescribed rank $k \in \mathbb{N}$, an $n \times n$ transformation matrix is parametrized by the product of two $k \times n$ and $n \times k$ sized matrices.

For least squares matching, we use the reduced rank regression outlined in the main text.

In all other respects, we utilize the same methodology and settings to set or train the stitching layer as described in Appendix A.3.

Figure 14 depicts further results for all the layers of the Tiny-10 architecture for this experiment.

A.11 Compute resources

We trained and evaluated approximately 20000 stitching layers overall. We used an internal cluster with GeForce 1080 Ti and GeForce 2080 Ti GPUs and dual Intel Xeon E5-2650 v4 CPUs in the machines. Each experimental run for a stitching layer with training and evaluation together used one (or a partial) GPU and generally finished under ten minutes for the Tiny-10 architecture, and under fifteen minutes for a 1-wide ResNet-20 on these machines.

B Further direct matching methods

B.1 Weighted mean squares matching of activations

In these experiments, we train the stitching layer to minimize the Weighted Mean Squared (WMS) objective. Our goal was to find a more or less simple method which sorts the activations, and assigns higher weights to the activations where a more precise matching is beneficial from the perspective of task performance. Roughly speaking, this method tries to identify more important features and put them into focus during the direct matching.

Given flattened activations $A, B \in \mathbb{R}^{n \times s}$ and a matrix of weights $W \in \mathbb{R}^{n \times s}$ we used Stochastic Gradient Descent to find the M which minimizes the WMS distance between AM and B:

$$\min_{M \in \mathbb{R}^{s \times s}} \|(AM - B) \circ W\|_F, \tag{4}$$

where $\circ$ is the Hadamard product $[X \circ Y]_{ij} = X_{ij} \cdot Y_{ij}$.

For the experiments listed below, we used the Tiny-10 architecture, the Adam optimizer with parameters $\beta_1 = 0.9$ and $\beta_2 = 0.999$, learning rate was set to 10^{-2}, batch size was 64 and we trained for 200 epochs.

In the following, we will present different choices for the unnormalized weight matrices W_u. In each case, we will normalize the weight matrix with entries in $[0, 1]$ as follows:

$$W = \frac{W_u - w_{\min} \mathbf{1_N} \mathbf{1_s^T}}{w_{\max}}, \tag{5}$$

where w_{min}, w_{max} denote the smallest and largest entries in W.

Gradient based weighting As the Class Saliency method points out Simonyan et al. [2014], the gradients of a network's outputs with respect to its inputs may contain valuable information about the importance of each part of the input. Our method is analogous, however we inspect the hidden activations instead of the inputs. In this experiment, we solved a weighted mean squares matching (4) where the weights were determined from the gradients in the network.

In the following, fix the network weights ϕ and the layer L, and denote by $T_k = T^k_{\phi,L}$ the task map from $\mathcal{A}_{\phi,L}$ to the kth class, and $\hat{T}_k = \hat{T}^k_{\phi,L}$ the task map from $\mathcal{A}_{\phi,L}$ to the kth class without the last non-linearity. Denote by $\partial_{ij} h = \partial_j h(x_i)$ the partial derivative of a real-valued function $h : \mathcal{A}_{\phi,L} \to \mathbb{R}$ on the ith datapoint according to the jth coordinate in the activation space $\mathcal{A}_{\phi,L}$.

We experimented with four variants of gradient based weighting; in each case, we took as unnormalized weights W^u a matrix of the form

$$W^u_{ij} = \partial_{ij} f(g_k),$$

where $f \in \{\sum_k, \mathrm{argmax}_k\}$ and $g_k \in \{T_k, \hat{T}_k\}$. In each case we solved the weighted mean squares matching problem (4) by taking as weight matrix W the normalization of such a weight matrix W^u. In particular, we considered the following variants:

- Gradients of summed output of Model 2 with respect to its matched activations:
$$W^u_{ij} = \partial_{ij} \sum_k T_k$$
- Gradients of the Model 2's output of the predicted class with respect to its matched activations:
$$W^u_{ij} = \partial_{ij} \operatorname*{argmax}_k T_k$$
- Gradients of summed output logits of Model 2 with respect to its matched activations:
$$W^u_{ij} = \partial_{ij} \sum_k \hat{T}_k$$
- Gradients of Model 2's output logits of the predicted class with respect to its matched activations:
$$W^u_{ij} = \partial_{ij} \operatorname*{argmax}_k \hat{T}_k$$

However, we found that overall these methods did not result in a consistent performance gain compared to the direct matching with unweighted least squares objective.

Activation based weighting Another approach is to take as unnormalized weights $W_u = B$, and use the corresponding normalized weight matrix W defined by (5). A simple way to emphasize higher values in this weight matrix is to take higher powers of each entry of W, i.e. use the weighting

$$W^{\circ n} = W \circ \ldots \circ W, \tag{6}$$

where $\circ$ denotes the (element-wise) Hadamard product.

We highlight some of these results for different values of n in Figure 15a. Our experiments show, that matching Tiny-10 architecture's layers this way results in significant performance gain compared to the Least Squares based direct matching. This indicates that matching the higher regime of activations accurately is more important than the lower regime.

Another way to force higher activation focused matching is simply to define a threshold T, and only match the activations above this threshold. In particular, this corresponds to a 0 – 1 weighting matrix

$$W = [1_{>T}(b_{ij})] \tag{7}$$

where $1_{>T}(x)$ denotes the indicator function, which serves as a threshold function. See Figure 15b for experiments with different thresholds T_i, defined by different percentiles $i \in \{10, 20\}$ of the activation values b_{ij}. We also tested higher threshold values defined by higher percentiles, which resulted in weaker performance. With this approach the activations below the threshold are not matched at all.

We also experimented with a slightly modified setup, where we define the weights as:

$$W_{ij} = 1 - 1_{<T}(b_{ij}) \cdot 1_{<T}([AM]_{ij}) = \begin{cases} 0, & [AM]_{ij} < T \text{ and } B_{ij} < T, \\ 1, & \text{else.} \end{cases} \tag{8}$$

This choice of weights achieves that if a target activation in B is above a certain threshold, the corresponding activation distance $(b_{ij} - [AM]_{ij})^2$ is always matched with weight 1. However, if the target activation in B is below the chosen threshold, then the direct matching only penalizes the corresponding distance if the matched activation $[AM]_{ij}$ exceeds the threshold. Informally, we don't care about the residuals as long as both activations are below the threshold. See Figure 15c for experiments with different thresholds T_i defined by different percentiles $i \in \{10, 20, 30, 40, 50\}$. The higher thresholds resulted in weaker performance.

Another version of this last experiment (with weights (8)), is to set the threshold $T = 0$, which is a reasonable choice because Tiny-10 uses ReLU activations. See Figure 15d for results.

C Further experiments

C.1 A sanity check for stitching

We measured the accuracy of stitching with task loss between N_i^1 and N_j^2, where N^1, N^2 are networks with the same architecture, trained on the same data but with different sample order, and N_i, N_j refers to the ith and jth layers of the network.

Stitched activations are required to have the same dimensionality, so we downsampled the higher dimensions with maxpooling before stitching. As upsampling without additional information is questionable, we always stitched from higher dimension (earlier layer) to lower dimension (later layer). Figure 16 shows the results measured on ResNet-20 architecture with width 1. We utilized the same methodology and settings to train the stitching layer as described in Appendix A.3.

C.2 Cross-task stitching, Feature visualization

The following experiments use the Lucent port[2] of the Lucid interpretability framework [Olah et al., 2017] to visualize how Model 2 channels are constructed from the linear combinations of Model 1 channels during direct matching and stitching. [3]

In the first setup, two Inception V1 networks trained on CelebA are stitched at various layers. The feature visualizations are shown in Figure 17. In the second setup, Model 1 is trained on ImageNet, and Model 2 is trained on CelebA. The feature visualizations are shown in Figure 18.

As seen in Tables 2a and 2b, the relative accuracies are consistently high, close to 100% in the case of the CelebA-CelebA stitchings. Even more notable is the case of the ImageNet-to-CelebA stitchings, where even the highest layers of the randomly initialized CelebA network can reuse the ImageNet features, with only a 3% relative drop in accuracy.

The accuracy of the original two CelebA models is 91.860% and 91.862%, respectively. Note that the supervised CelebA task consists of 40 binary classification subtasks, and reported accuracy is an average (across the validation set) of averages (across subtasks). The tasks are unbalanced, hence the accuracy of the majority baseline is already quite high, 80.5% (or 87.63% relative to Model 2 accuracy), which is important to keep in mind when interpreting the values in Table 2. Nevertheless, it is apparent that stitchings perform nearly perfectly when done between models trained on the same dataset, and perform well even across datasets, especially when done at the lower layers.

layer	Least Squares	Frankenstein
3a	99.45%	99.88%
3b	98.73%	99.86%
4a	99.41%	99.85%
4b	99.21%	99.84%
4c	99.05%	99.87%
4d	98.80%	99.91%
4e	98.88%	100.01%
5a	99.98%	100.07%
5b	99.92%	100.08%

(a) Stitchings between two CelebA networks with different random initializations.

layer	Least Squares	Frankenstein
3a	97.72%	99.77%
3b	95.90%	99.62%
4a	97.73%	99.41%
4b	96.74%	99.22%
4c	95.27%	98.88%
4d	93.45%	98.42%
4e	94.37%	98.21%
5a	95.75%	97.49%
5b	95.57%	97.02%

(b) Imagenet to CelebA stitchings.

Table 2: Inception V1 stitchings, relative accuracies.

[2] `https://github.com/greentfrapp/lucent`

[3] Note that a limitation of backpropagation-based feature visualization is that even though the visualization presents a pattern that activates the channel, it can be activated by very different patterns as well.